

1. What to write in the observation book

Do not copy the full theory pages. For each question, use:

Aim

Concept

Procedure/Commands

Configuration

Execution Flow

Observed Output

Result

Replace sample outputs with the outputs you actually obtain.

Observation Book: Question 1

Aim

To configure Nginx to serve a static HTML webpage using IP-based virtual hosting on the secondary loopback address `127.0.0.2`.

Concept

IP-based virtual hosting selects an Nginx server block using the destination IP address and port. A secondary loopback IP was assigned to the local system, and Nginx was configured to listen specifically on `127.0.0.2:8080`.

Procedure

1. Assigned the secondary loopback address:

```
sudo ip addr add 127.0.0.2/8 dev lo
ip addr show lo
```

2. Created the document root and HTML file:

```
sudo mkdir -p /var/www/ip_site
sudo nano /var/www/ip_site/index.html
```

3. Created `/etc/nginx/sites-available/ip_site.conf`:

```
server {
listen 127.0.0.2:8080;
server_name _;

root /var/www/ip_site;
index index.html;

location / {
try_files $uri $uri/ =404;
}
}
```

4. Enabled and applied the configuration:

```
sudo ln -s /etc/nginx/sites-available/ip_site.conf </span>
/etc/nginx/sites-enabled/ip_site.conf
sudo nginx -t
sudo systemctl reload nginx
```

5. Verified the webpage:

```
curl http://127.0.0.2:8080
```

Execution flow

```
Client request to 127.0.0.2:8080
→ Nginx matches listen 127.0.0.2:8080
→ /var/www/ip_site is selected
→ index.html is read
→ static HTML response is returned
```

Observed output

```
IP-Based Virtual Hosting is working
This page is served through 127.0.0.2 on port 8080.
..
```

`sudo nginx -t` reported successful configuration syntax. The browser/`curl` displayed the content stored in `index.html`. An incoming request entry was generated in the Nginx access log.

Why this output occurred

The HTTP request used destination IP `127.0.0.2` and port `8080`. This matched the `listen 127.0.0.2:8080` directive, so Nginx used `/var/www/ip_site` as the document root and returned `index.html`.

Result

Nginx was successfully configured to serve a static webpage through IP-based virtual hosting using `127.0.0.2:8080`.

Observation Book: Question 2

Aim

To configure Nginx as a reverse proxy for a dynamic JSP application executed by Apache Tomcat.

Concept

Nginx receives the client request on port `8080`, but JSP is executed by Tomcat on port `8888`. Nginx therefore forwards the request to Tomcat using `proxy_pass`. Tomcat processes the JSP and returns the generated response through Nginx.

Procedure

1. Created the JSP application under Tomcat's deployment directory:

```
sudo mkdir -p /var/lib/tomcat10/webapps/sample
sudo nano /var/lib/tomcat10/webapps/sample/index.jsp
```

2. Added the JSP page:

```
<%@ page contentType="text/html; charset=UTF-8" %>
<html>
```

```
<body>
<h1>Nginx to Tomcat Reverse Proxy is working</h1>
<p>JSP executed successfully.</p>
<p>Server time: <%= new java.util.Date() %></p>
</body>
</html>
```

3. Verified Tomcat directly:

curl <http://127.0.0.1:8888/sample/>

4. Added the following hosts-file entry:

127.0.0.1 jspsite.local

5. Created the Nginx server block:

```
server {
listen 8080;
server_name jspsite.local;

location / {
proxy_pass http://127.0.0.1:8888/sample/;
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
}
}
```

6. Enabled, validated, and reloaded Nginx:

```
sudo ln -s /etc/nginx/sites-available/jsp_site.conf </span>
/etc/nginx/sites-enabled/jsp_site.conf
sudo nginx -t
sudo systemctl reload nginx
```

7. Verified through Nginx:

curl <http://jspsite.local:8080/>

Execution flow

Browser

→ jspsite.local resolves to 127.0.0.1

- Nginx receives request on port 8080
- proxy_pass forwards request to Tomcat:8888/sample/
- Tomcat/Jasper processes index.jsp
- dynamic HTML response is generated
- response returns through Nginx
- browser displays the page

Observed output

Nginx to Tomcat Reverse Proxy is working

JSP executed successfully.

Server time: [actual date and time displayed during execution]

The changing server time confirmed that the JSP was processed dynamically rather than returned as an unchanged static file.

Why this output occurred

The Nginx server block matched `jpsite.local`. The `proxy_pass` directive forwarded the request from port `8080` to the sample application running in Tomcat on port `8888`. Tomcat processed the JSP expression and returned generated HTML to Nginx.

Result

Nginx was successfully configured as a reverse proxy, and Tomcat successfully executed and served the dynamic JSP page.

Final lab checklist

- ☐ Correct IP/name mapping
- ☐ Correct document/application folder
- ☐ Correct Nginx server block
- ☐ Correct PHP socket or Tomcat port
- ☐ Symbolic link created in sites-enabled
- ☐ `sudo nginx -t` succeeds
- ☐ Nginx reloaded
- ☐ Browser/curl output recorded

- ❑ Access/error logs checked
- ❑ Actual screenshots/output attached if required
- ❑ `sudo lab-reset` executed after completing both exercises

The manual makes `sudo lab-reset` mandatory after completing the Lab 1 exercises.

[\[CCD\(3130\) Lab Manual | PDF\]](#)